Retrieval-Augmented Vulnerability Detection via Code Property Graphs

May 2026

What Is a Vulnerability?

NIST SP 800-53 / CWE Definition

A **vulnerability** is a weakness in a system that can be exploited by a threat to gain unauthorized access or cause harm.

Example:
Use-After-Free

The program references memory after it has been freed, leading to undefined behavior.

```
struct sock *alloc_socket(
    struct socket *sock) {
  struct sock *sk;

  sk = sk_alloc();        // allocate

  sock->sk = sk;          // alias created

  if (!chan_create()) {
    sk_free(sk);          // free original
    return NULL;     // BUG: alias dangling!
  }
  return sk;
}
```

Tracing the Vulnerability (1/4): Allocation

```
struct sock *alloc_socket(
    struct socket *sock) {
  struct sock *sk;

  sk = sk_alloc();          // allocate

  sock->sk = sk;            // alias created

  if (!chan_create()) {
    sk_free(sk);            // free original
    return NULL;
  }
  return sk;
}
```

- `sk` **is allocated** — a new object is created on the heap

Tracing the Vulnerability (2/4): Alias Creation

```
struct sock *alloc_socket(
    struct socket *sock) {
  struct sock *sk;

  sk = sk_alloc();          // allocate

  sock->sk = sk;            // alias created

  if (!chan_create()) {
    sk_free(sk);            // free original
    return NULL;
  }
  return sk;
}
```

- `sk` is allocated
- `sock->sk` **becomes an alias** — two pointers now reference the same object

Tracing the Vulnerability (3/4): Free

```
struct sock *alloc_socket(
    struct socket *sock) {
  struct sock *sk;

  sk = sk_alloc();          // allocate

  sock->sk = sk;            // alias created

  if (!chan_create()) {
    sk_free(sk);            // free original
    return NULL;
  }
  return sk;
}
```

- `sk` is allocated
- `sock->sk` becomes an alias
- **On error,** `sk` **is freed** — the object is deallocated

Tracing the Vulnerability (4/4): Dangling Pointer

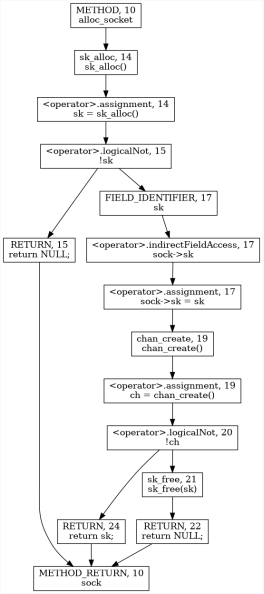
```
struct sock *alloc_socket(
    struct socket *sock) {
  struct sock *sk;

  sk = sk_alloc();          // allocate

  sock->sk = sk;            // alias created

  if (!chan_create()) {
    sk_free(sk);            // free original
    return NULL;      // BUG: alias dangling!
  }
  return sk;
}
```

- sk is allocated
- sock->sk becomes an alias
- On error, sk is freed
- sock->sk **still points to freed memory!**

Pattern:
Free without nullifying all aliases
⇒ **Use-After-Free**

This pattern leaves a structural fingerprint in the code

Code Property Graph: Full Joern Output

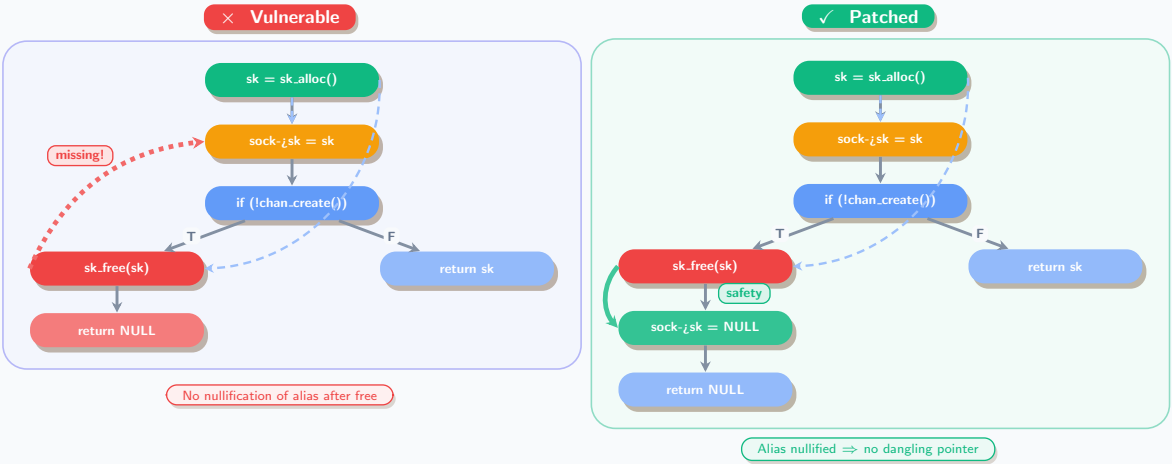


What the CPG encodes:

- **AST** — syntax structure
- **CFG** — execution order
- **DDG** — data flow (def → use)
- **CDG** — control dependencies

Full graph is detailed — let's simplify to the vulnerability-relevant subgraph.

Structure Reveals the Vulnerability



Takeaway: The structural difference between vulnerable and patched code is a single edge/node — vulnerabilities leave fingerprints in the graph.

Motivation: Why Not Just Train a Graph Classifier?

Prior work exploits the same insight — code structure reveals vulnerabilities:

- Train GNNs / graph kernels on labeled CPGs or PDGs
- Learn to classify subgraphs as vulnerable vs. safe
- Devign, ReVeal, IVDetect, ...

But these approaches hit a wall:

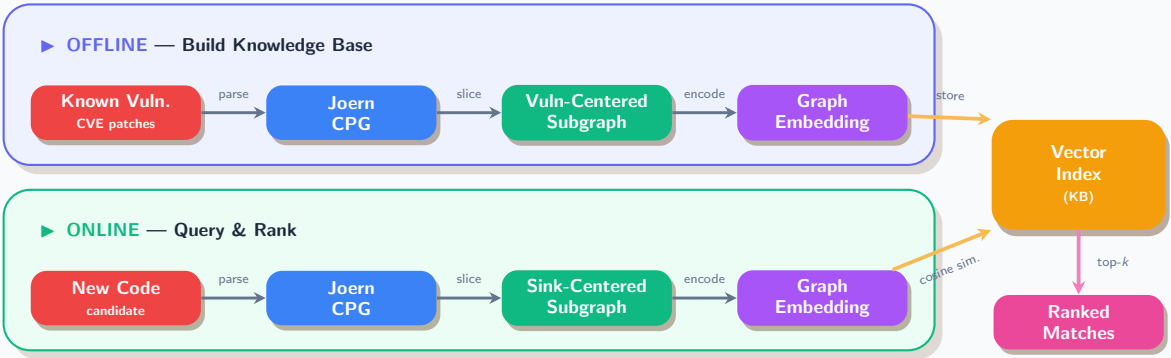
- Require large labeled datasets (expensive to curate)
- Struggle to generalize to *unseen* vulnerability types
- Treat each vulnerability independently — no knowledge reuse

Our alternative: Instead of *classifying*, **retrieve**.
Store known vulnerability patterns as graph embeddings in a knowledge base, then rank new code by structural similarity — no retraining needed.

Research Questions

RQ1: How effective are graph-based embeddings at retrieving relevant vulnerability fix patterns from the knowledge base?
RQ2: What are the trade-offs of using Graph-RAG in the vulnerability patching pipeline?

System Architecture: Graph-RAG Pipeline



Key idea: Identical pipeline for both tracks — only the *input* differs. **Offline** indexes *known* vulnerability patterns; **Online** queries *unseen* code against them.

Feasibility Study: AutoPatch Dataset (n=225)

Goal: Validate the concept using a controlled dataset and define baselines.

AutoPatch Dataset:

- 75 high-severity CVEs (57 Linux Kernel, 10 Chromium)
- 5 LLMs re-implement each CVE $\rightarrow$ 375 code snippets
- Augmented: 75 vulnerable + 75 patched variants per CVE
- Trivial transforms (whitespace, comments, dead code)
- Non-trivial transforms (renamed vars, added library calls)

Embeddings compared:

1. Structure-only: NetLSD + WL + GIN (combined)
2. Semantic baseline: CodeBERT

Key Question:
Can structural graph embeddings capture vulnerability patterns as effectively as pretrained CodeBERT?

Hypothesis: Structural graph representations are as effective as semantic code models for vulnerability pattern matching, and allow high-quality retrieval.

Evaluation Metrics

Hit@1 (H@1) Fraction of queries where the correct match is ranked first.

Hit@5 (H@5) Fraction of queries where the correct match appears in the top 5.

MRR Mean Reciprocal Rank — average of $\frac{1}{\text{rank}}$ across all queries. Rewards higher placements more.

CWE Recall (CWE R.) Fraction of distinct CWE types in the index that are correctly retrieved at least once.

Why these metrics?

- H@1 / H@5 measure practical utility — does the right pattern surface quickly?
- MRR captures ranking quality beyond just top-1.
- CWE Recall measures coverage — can the system retrieve *diverse* vulnerability types, not just frequent ones?

Embedding Pipeline: From Graph to Vector

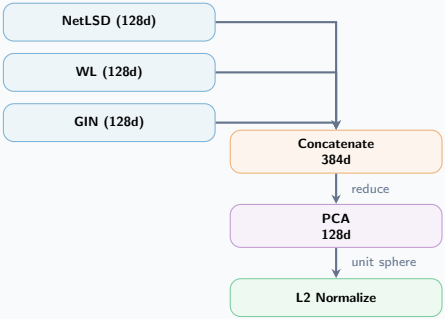
Structure-Only Embedders (no training required):

Embedder	Dim	Mechanism
NetLSD	128	Heat kernel trace on graph Laplacian spectrum (100 timescales 10^{-2}–10^{2})
WL	128	Weisfeiler-Lehman color refinement (4 iters) $\rightarrow$ sum pool $\rightarrow$ linear proj.
GIN	128	3-layer Graph Isomorphism Network (random weights, frozen) $\rightarrow$ mean+sum readout

Semantic Baseline:

Embedder	Dim	Mechanism
CodeBERT	768	Pretrained transformer; CLS token on linearized code

Combined Pipeline:



PCA: fit on index set, retains >95% variance.
All final vectors: **128d**, L2-normalized.

Node features: 11-class CPG node type (one-hot) — METHOD, CALL, IDENTIFIER, LITERAL, RETURN, CONTROL_STRUCTURE, ...

Phase 1: Feasibility — AutoPatch Dataset (n=225)

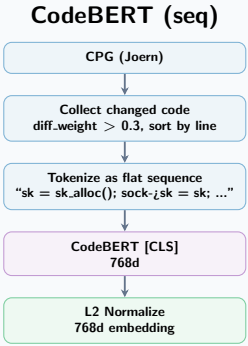
Goal: Validate that structural embeddings match CodeBERT performance.

Embedder	H@1	H@5	MRR	CWE R.
Structure-Only				
Combined	**0.8904**	**0.9589**	**0.9162**	**0.4978**
GIN	0.7260	0.8767	0.7879	0.4427
WL	0.6986	0.7945	0.7363	0.4386
CodeBERT pretrained				
CodeBERT (baseline)	0.1533	0.2200	0.1826	0.2933
CodeBERT (seq)	0.7534	0.8630	0.7927	0.4121
CodeBERT (pat)	0.7123	0.8356	0.7677	0.4147

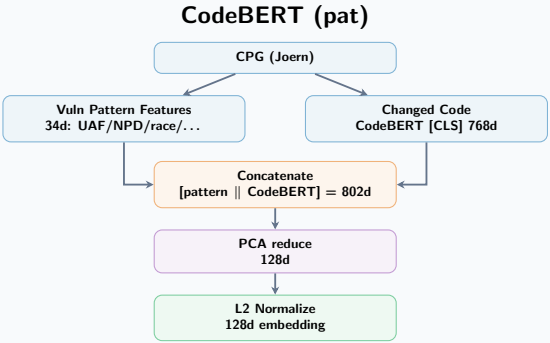
Source: `experiments/output/checkpoint5/20260505_141347_experiment_f59ab1/results.json`

Key Finding: Combined structural embeddings (89% Hit@1) outperform CodeBERT (75%).
RQ1: validated — structural *graph* representations are *effective* for vulnerability pattern retrieval in a synthetic dataset.

CodeBERT Embeddings — Sequence vs. Pattern



Text-only: structure is **discarded**.



Fuses **graph structure** with semantics.

Phase 2: Generalization — CVEfixes Dataset

Question: Does the approach generalize to real-world vulnerabilities?

Goal

CVEFixes datasets is large dataset with real-world CVEs collected from the U.S. National Vulnerability Database, patches, and code changes.

Metric	Value
CVEs	11,873
Fix commits	12,107
File changes	51,342
Method changes	277,948
Repositories	4,249

Sampling criteria

Structurally distinshable CWEs (e.g. UAF vs. OOB) to test pattern matching, not just code similarity

Balanced superclass(CWE) up to 20 entries and subclasses (CVE) to ensure retrieval feasibility

Parameter	Value
Total pairs	136
Index set	109
Query set	27
Unique CVEs (index)	60
Unique CVEs (query)	25
CWE classes	7

Phase 2: Pipeline Verification Results

Same-CWE Retrieval (n=109 index, 27 query)

Embedder	Hit@1	Hit@5	MRR	CWE Recall
GIN	0.259	0.778	0.315	0.202
Combined	0.111	0.778	0.223	0.189
CodeBERT (baseline)	TODO?			
CodeBERT (seq)	**0.556**	**0.926**	**0.555**	**0.264**
CodeBERT (pat)	**0.519**	**0.889**	**0.549**	**0.276**

Source: `cvefixes_experiments/output/pipeline_verification/results.json`

RQ1 validated on real-world dataset with same-CWE retrieval up to 93% at k=5.

Per-CWE Breakdown (CodeBERT Pattern, CWE Hit@k)

CWE	N	@1	@5	@10
Integer Overflow (190)	4	75%	100%	100%
Input Validation (20)	4	75%	75%	100%
Resource Consump. (400)	3	67%	100%	100%
Use After Free (416)	4	50%	100%	100%
NULL Ptr Deref (476)	5	40%	80%	80%
Race Condition (362)	3	33%	100%	100%
OOB Write (787)	4	25%	75%	75%
Overall	**27**	**51.9%**	**88.9%**	**92.6%**

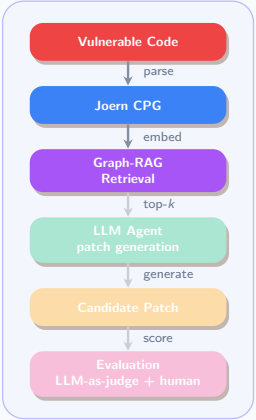
Source: same file (CodeBERT Pattern cell)

Highlights areas for improvement in embedding design or slicing strategy.

Next Steps: Research Question 2

RQ2: What are the trade-offs of using Graph-RAG in the vulnerability patching pipeline?

1. Integrate the retrieval layer with a small, open-source LLM and validate the end-to-end patching pipeline.
2. Run comparative experiments against baseline approaches, including zero-shot open-source models and AutoPatch.
3. Refine the patch generation logic based on experimental results and finalize the implementation.



Appendix: CPG Deep Dive

The Code Property Graph encodes four types of relationships:

- AST: Abstract syntax tree (structure of code)
- CFG: Control flow graph (which statements execute in sequence)
- DDG: Data dependency graph (variable definitions and uses)
- CDG: Control dependency graph (which statements depend on conditionals)

For vulnerability detection:

- Look for patterns in DDG (data flow anomalies)
- Combine with CFG to understand execution order
- Use CDG to understand conditional branches (e.g., error handling)

Joern exposes all four via traversals; KB retrieval compares the combined structure to find similar vulnerabilities.

Appendix: Joern Query - Finding the Pattern

Goal: Find `free()` calls with dangling aliases.

```
val uafCandidates = cpg.call.name(".*free.*")
  .map { freeCall =>
    val freedVar = freeCall.argument(1).code
    val aliases = freeCall.method.assignment
      .where(_.source.isIdentifier.name(freedVar))
      .target.code.l
    val freeLineNo = freeCall.lineNumber.get
    val nullified = freeCall.method.assignment
      .where(_.lineNumber.map(_ > freeLineNo))
      .target.code.l
    val dangling = aliases.filterNot(nullified.contains)
    (freeCall, dangling)
  }
```

Output: `[UAF] sk_free(sk) | dangling: sock->sk`